



AI時代のテスト自動化について

AIエージェントに安全に開発を任せるための
ハーネスエンジニアリング

第1回 AI勉強会 2026年6月26日

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

はじめに：勉強会開催の理由



1. 市場の高まる「AIニーズに応えられる会社」へ成長していきたい。
2. 今後より一層、AIを使いこなせるエンジニアの「市場価値」は高まる。
3. 「AIスキル向上」のプログラムの第1歩として、今回の勉強会を開催。

はじめに：勉強会のゴール



1. AIを試して終わりにせず、開発現場で「実際に使える」ようになる。
2. テスト自動化を、「AIが作ったものを確認する仕組み」と理解する。
3. 完璧を目指さず、「明日から試せる小さなこと」を出来るようになる。

はじめに：講師紹介



伊藤 元さん

専門領域

AI駆動開発

WEBアプリケーション開発、スマートフォンアプリ開発

経験

20年近くシステム開発に従事。

7年間のSES企業での勤務を経て、楽天株式会社へ転職

その後、個人事業主として、様々な業界のシステム開発を遂行。

要件ヒヤリング～システム設計、開発、テスト、運用を一人で担当。

自社サービスの開発も実施。

実績

テーブルカードゲーム向けの価格サイトの開発運用/テーブルカードゲーム向けのイベントカレンダーアプリの開発運用

EC業界向けの決済連携システム開発や、不正検知システム開発、会員プラットフォームの開発 等

一言

AIの活用によって、エンジニアリングのスピードやレベルが格段に上がることを実感しております。是非、一緒に学んでいけたらと思います！

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

AI開発の全体像：まず理解してほしいこと



AIは高精度

実装もテストのコードも、優秀なAIエージェントを使えば、想像以上にスピーディーに進められる。

それでも根拠は必要

システム開発では、AIが「できました」と言っただけでは、本番に出していいとは判断できない。

テストは土台になる

既存の挙動を固定しておけば、変更した後も問題ないかを機械的に確認することができる。

人間は判断材料を設計

コードレビュー、センサー、作業記録等から、「次に進めて良いか」を人間が決める仕組みを作る。

| AI開発の全体像：ここで質問です。



以下の依頼について、
AIが修正した後、何を根拠にリリース OKと判定すれば良いか？

依頼

既存の注文見積りAPIで、
送料無料条件を5,000円以上から7,000円以上へ変更してください。

| AI開発の全体像：ここで質問です。



自分が関わっていない既存プロジェクトなら、
どこまで確認したら OKと判定すれば良いか？

- ✓ 既存の機能は壊れていないか？
- ✓ 仕様書と実装は一致しているか？
- ✓ AIによる修正箇所は保守できる形か？
- ✓ 判断の理由は次回に残るか？

AI開発の全体像：AIを入れるときの課題



テストが不十分

自動テストが少なく、人の手作業での確認や、担当者の経験・記憶に頼ってしまっている。

仕様が分散

仕様に関する情報が、ドキュメント・Slackでのやり取り・コード・運用時の判断など、バラバラの場所に散らばっている。

AIの作業スピードが速い

AIはコードの変更をすぐに作れるが、見た目が良くても「なぜそれで正しいと言えるか」の裏付けが弱いまま進んでしまいやすい。

確認が追いつかない

AIが速く作業を進めても、レビュー担当がそのすべてを一つずつ丁寧に確認しようとする、そのスピード差がそのままレビュー担当の負担(確認待ち・確認疲れ)になってしまう。

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

テスト自動化：テスト自動化とは



	従来の見方	今回の見方
テスト	ゴール	判断のためのセンサー
テスト自動化	人間が書くテストコードをAIやツールで速く作る。	AIがコードを変更した後、人間が受け入れ判断できる材料を自動で集める。

テスト自動化：AIエージェントとは



これまでのAI（プロンプト）のイメージ

質問に答える、文章を作る、など「1回の指示に1回答える」イメージ。

AIエージェント

目標だけ与えたら、その達成に向けて 自分で次の行動を選び、実行する。あらかじめ決められた手順に従う従来のソフトウェアとは異なり、AIエージェントは過去のデータをもとに次取るべき行動を自分で判断し、人がずっと監視しなくても実行が可能。

テスト自動化：AIエージェントの課題



AIエージェントは作業を肩代わりして便利であるが、
リリース判断は人間の責任

出来る事	残るリスク	人間が決める事
読む	古いドキュメントを信じる。	根拠の優先順位
直す	範囲外まで修正してしまう。	修正箇所の境界線
テストを書く	失敗しにくいテストばかり書いてしまう。	代表値と不変条件
実行して修復	テストを通すためだけの雑な修正を行う。	受け入れ条件

テスト自動化：ハーネスエンジニアリングとは



AIの自由度を消すものではなく、受け入れ可能な範囲へ導くもの

ハーネス

馬具・装具という意味。足場、土台とも。

「力の強いものを制御しながら、目的の方向に動かす仕組み」

ハーネスエンジニアリング

AIが正しく安全に動くための「足場」（周りの仕組み）を作り込むエンジニアリング

作業前

前提、制約、変更範囲を渡す。

作業後

テスト、センサーで結果を見る。

判断後

理由と残リスクを記録する。

テスト自動化：ハーネスエンジニアリングとは 人

AIを信じる/信じないではなく、AIが出した変更を判断できる形にする

観点	内容
目的	コーディングエージェントの結果を信頼しやすくするため、作業前のGuidesと作業後のSensorsを設計する。
動き	望ましくない出力を先に防ぎ、出た問題はセンサーで検出して自己修正へ回す
人間の役割	すべてを読むのではなく、GuidesとSensorsを育て、重要判断へ集中する。
今回のデモ	CR、AGENTS.md、テスト、policy-boundary、worklogで小さく体験する。

出典：Birgitta Böckeler氏がMartin Fowlerサイトの記事で、coding agent利用者向けにまとめた考え方。

テスト自動化：ハーネスエンジニアリングとは



押さえておく用語

用語	意味	今回の例
CR	Change Request 変更依頼、背景、受け入れ条件、 対象外をまとめる入口	送料無料を 7,000円以上へ変更
AGENTS.md	Codex向けの作業規約 CLAUDE.md / GEMINI.mdと 同じ立ち位置	責務境界、テスト、 記録ルール
ADR	Architecture Decision Record 設計判断と理由を短く残す記録	閾値変更や policy-boundaryの判断
characterization test	変更前の仕様で、 現在の挙動を固定するテスト	4,999/5,000（円）境界 を固定

テスト自動化：ハーネスエンジニアリングとは



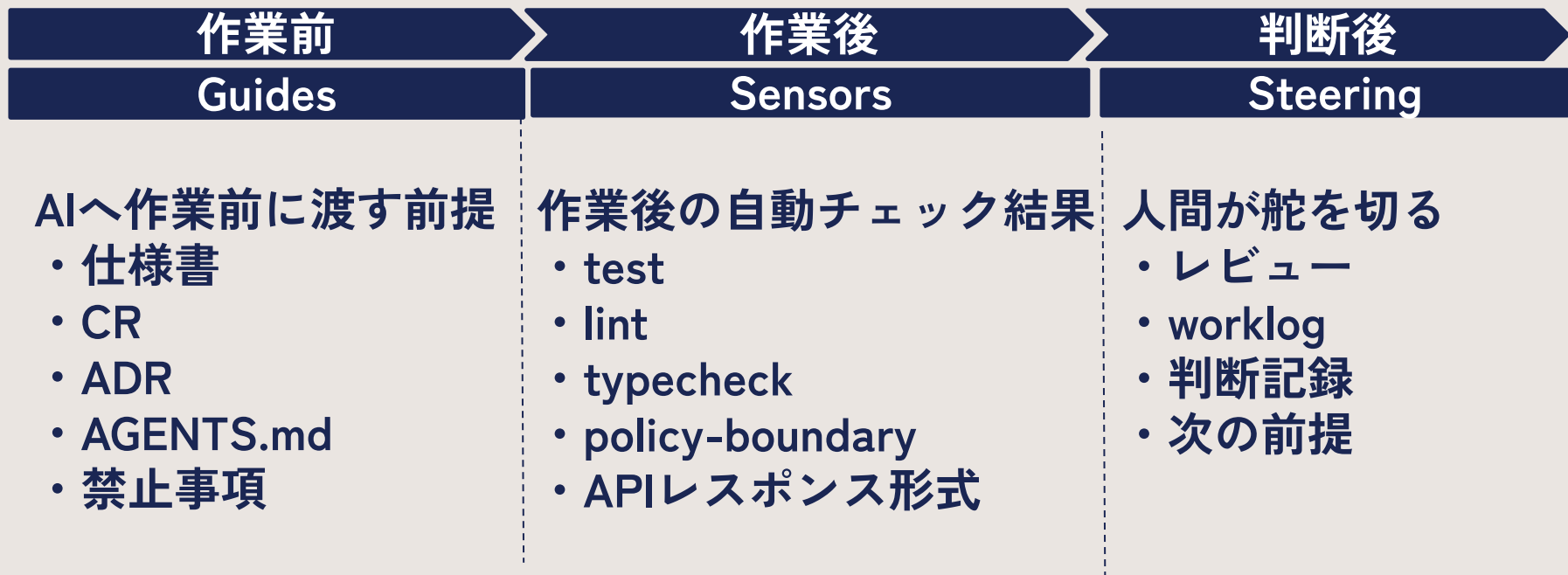
押さえておく用語

用語	意味	今回の例
Guides	AIへ作業前に渡す前提情報	仕様書 ・ CR ・ ADR ・ AGENTS. md
Sensor	機械的な確認。 赤/緑で受け入れ前の問題を発見	test、 policy-boundary、 API response
Steering	人間が判断し、 その判断を次の前提へ反映	diff(差分), worklog(作業 記録), completion-report(完了報 告)
policy-boundary	ルールの境界値チェック	6,999円→送料有 7,000円→送料無

テスト自動化：Guides/Sensors/Steering



今日のデモは、この3つに、少しずつ要素を追加していく流れです。



| ここまでのまとめ



- ✓ AIやAIエージェントによって、高スピード、高精度で開発が可能となったが、様々な課題もある。リリース可否は、依然として人間の判断が必要とされている。
- ✓ そのため、テスト工程を「判断のためのセンサー」、テスト自動化を「人間が受け入れ判断できる材料を自動で収集」と位置づけることで、AIのメリットを享受しながら、AIの課題を解決する。
- ✓ そのために必要なものが「ハーネスエンジニアリング」
Guides/Sensors/Steeringの3つをベースに、AIが安全に動くための土台を構築する。

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

デモ：デモの全体像



	出来る事	不足してる事
Demo0 開始前	AIへ依頼する	根拠、テスト、制約、記録
Demo1 変更	機能変更は出来たかも	受け入れる証拠
Demo2 挙動	既存挙動をテストで固定	構造保証、Guides、記録
Demo3 構造	送料無料ルールの境界を検知	CR、Steering、出口
Demo4 候補	リリース候補としてレビュー可能	CI（継続的インテグレーション）、本番確認、運用
Demo5 介入圧縮	途中介入を減らせる	センサー成熟、停止条件

デモ：デモ題材と開始時点の足りないもの



Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

```
project/  
src/controllers/orderController.js  
src/services/orderEstimateService.js  
src/services/promotionService.js # threshold (閾値) 重複  
src/config/pricing.js           # threshold (閾値) 定義  
docs/specs/order-estimate.md    # 3,000円の古い記述
```

内容	詳細	開始時点
既存挙動の固定	characterization test	ない
新仕様の受け入れ条件	「OK」とする判断基準	ない
構造チェック	コードの形式や書き方をチェック する仕組み(lint・typecheckなど)	ない
AIへの作業規約	AGENTS.mdなど	ない
判断記録	ADR・worklogなど	ない
人間が確認・判断する出口		未定

デモ：デモ題材と開始時点の不足物



Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

```
$ prompt  
送料無料の条件を5,000円以上から7,000円以上に変更してください。
```

```
$ rg "5000|FREE_SHIPPING|threshold" src docs
```

```
src/config/pricing.js  
src/services/promotionService.js  
docs/specs/order-estimate.md
```

出来たこと	不足してること
AIへ変更依頼	証明と記録がない

✓ ポイント

AIが間違えるかではなく、変更後に「OK」と判断する根拠が何かです。

デモ：正しく直っても、テストなしでは証明不可



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

```
$ curl subtotal=6000
```

```
{"subtotal":6000,"shippingFee":500,  
"freeShippingThreshold":7000,  
"freeShippingRemaining":0,  
"message":"送料無料"}
```

出来たこと

手動確認は一部出来た。

不足してること

リリース判断はまだ出来ない。

✓ ポイント

AIが、仮にThresholdの箇所を重複分も含めて修正したとしても、
testsなし/APIレスポンス形式チェックなし/構造チェックなし/判断記録なし
では、今回の成功が、次回も成功するという根拠にはならない。

デモ：まず観点を出し、既存挙動を固定



Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

そのまま貼るプロンプト

現在の注文見積もりAPIの挙動を調査し、
既存挙動を固定するcharacterization testを作成してください。
いきなりテストを書かず、先に観点表を出してください。
観点: 公開API / 境界値 / 不変条件 / エラーケース / docs差分
各観点について、テスト化する/しない、理由、確認方法を表にし、
採用した観点だけをテストにしてください。

出来たこと	不足してること
既存挙動を再確認	新仕様と構造保証はまだない

✓ AIの出力結果

先に出力：観点表: 何をテスト化し、何を記録に回すか。

生成物 : tests/*.test.js / tests/testHelper.js 境界値、不変条件、
API response shape、不正入力

デモ：characterization test



Demo0
開始前

- ✓ 今の挙動をテストで固定できたので、次は、挙動を変えず送料無料判定処理を1か所にまとめる。
- ✓ 最初にしたcharacterization testは、あとで3つの役割に分かれる。

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

種類	扱い	今回の例
残す	今後も変わらない挙動として、回帰テストとして残す	レスポンス形式、不正な入力への対応、合計金額の計算
書き換える	新仕様の受け入れテストへ変更	4,999円/5,000円の境界 → 6,999円/7,000円の境界
記録へ移す	worklogやADRへ残す	仕様書(docs)が古いまま、閾値が2か所に重複していた

デモ：送料無料ルールの境界をセンサーで検知



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

そのまま貼るプロンプト

送料無料判定の責務を整理してください。

条件:

- API挙動は変えない / 既存testは全て通す
- 判定・閾値取得・残額計算を1つのpolicy moduleへ集約
- orderEstimateServiceとpromotionServiceで閾値判断を分裂させない
- 境界ドキュメントとpolicy-boundary sensorを作る
- npm run sensorsに登録して実行する

✓ AIが修正するファイル

ファイル	簡単な説明
orderEstimateService.js	送料無料の判定ロジックを取り除き、新しいfreeShippingPolicy.jsを使うよう修正
promotionService.js	重複してる閾値の判定ロジックを取り除き、同じくfreeShippingPolicy.jsを使うよう修正

デモ：送料無料ルールの境界をセンサーで検知



✓ AIが新規に作成したファイル

Demo0 開始前	ファイル	簡単な説明
Demo1 変更	freeShippingPolicy.js	送料無料の判定ロジックを1か所にまとめる新しい専用ファイル
Demo2 挙動	order-estimate-boundaries.md	閾値についてまとめた説明ドキュメント
Demo3 構造	check-policy-boundary.js	閾値が正しく機能しているかを自動でチェックするスクリプト
Demo4 候補	run-sensors.js	上記のチェックなどをまとめて実行するためのスクリプト
Demo5 介入圧縮	package.json	npm run sensorsでチェックを実行できるよう、コマンドの登録を追加

デモ：送料無料ルールの境界をセンサーで検知



✓ 完了条件

Demo0 開始前	条件	簡単な説明
	node --test green	Node.jsの標準テストが全部成功する
Demo1 変更	policy-boundary green	閾値のチェックが成功する
Demo2 挙動	API response green	APIの返り値のチェックが成功する
Demo3 構造	出来たこと	不足してること
Demo4 候補	ルールの境界を検知できる	正式な変更依頼(CR)と 最終判断する出口がまだない
Demo5 介入圧縮	✓ ポイント 7,000円への変更はまだしておらず、今回は構造の整理だけ。 動けばOKではなく、今後保守できる形を見通して作業を行う。	

デモ：ここまでで入力したプロンプトと作成物



Demo0 開始前	段階	プロンプトで指示したこと	AIが生成したもの
Demo1 変更	Demo2	観点表を出し、 採用した観点だけをテスト化	tests/*.test.js / tests/testHelper.js
Demo2 挙動	Demo3	送料無料判定を 1つのpolicy moduleへ集約	freeShippingPolicy.js/ orderEstimateService.js(修正)/ promotionService.js(修正)
Demo3 構造	Demo3	責務境界を文書化し、 policy-boundary sensorを登録	order-estimate-boundaries.md / check-policy-boundary.js/ run-sensors.js/ package.json
Demo4 候補			
Demo5 介入圧縮			
✓ ポイント ここまでで、既存挙動のテストと送料無料ルールの境界検知センサーが揃う。			

デモ：人間が用意しておく Guides



	ファイル	誰が用意	中身
Demo0 開始前	CR	人	背景: 配送コスト上昇/ 受け入れ条件: 6,999円は送料500円、7,000円は送料0円/対象外: UI・税・DB
Demo1 変更			
Demo2 挙動	AGENTS.md	人/チーム	作業範囲はCRの内容だけ/API response形式は変えない/既存テストを削除してごまかさない/sensorsを実行
Demo3 構造	order-estimate.md	既存Doc	最終更新:: 2024-11/書かれている送料無料の金額:3,000円/現実と異なる可能性あり
Demo4 候補	ADR-0001	既存doc	controllerはHTTP入出力だけ / 計算はserviceが担当 / 閾値と送料はconfigに記載 / policy (ルール) 境界を守る
Demo5 介入圧縮	✓ ポイント AIが勝手に作るものではなく、人が入口として置いて、AIに読ませる前提。		

デモ：Guidesを読ませて、リリース候補へ



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

そのまま貼るプロンプト

AGENTS.mdを読んだ上で、
CR-2026-06-26-free-shipping-thresholdを実装してください。

- 既存testを残す/書き換える/記録へ移す
- 4999/5000を6999/7000へ書き換える
- spec / ADR / worklogを更新する
- 全センサー緑で完了報告する

AIに読み込ませるもの

CR
AGENTS.md
order-estimate.md
ADR-0001

AIが更新するもの

src/config/pricing.js
order-estimate.md

AIが生成するもの

境界値テスト、ADR、worklog

出来たこと

作業前提と確認が揃った

不足してること

出口レビューは人間が行う

デモ：古い挙動テストを新仕様の受け入れテストへ



Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

CONFIDENTIAL

RED: expected free shipping at 5000, got 500

rewrite:

- 4999 / 5000 -> 6999 / 7000 #境界値のテストを新しい閾値に
- response shape stays #レスポンス形式のテストは変更しない
- invalid input stays #不正入力の対応テストは変更しない
- total = subtotal + shippingFee stays #合計金額確認テストは変更しない

| sensor | after fix |

| test | green |

| policy-boundary | green |

| APIレスポンス形式 | green |

✓ ポイント

Demo 1は人間がcurlで手探りに確認していたが、今回はテストが自動で赤を出して問題を教えてくれる。AIはその内容を説明し、すべて緑となるようにする。

デモ：人間とAIと機械の責務を分ける



Demo0 開始前	担当	見るもの	判断
Demo1 変更	機械	test / lint / typecheck / policy-boundary / APIレスポンス形式	条件に合わなければ赤
Demo2 挙動	AI	赤の原因、関連ファイル、修復方針	修正して再実行
Demo3 構造	人間	CR、diff、残リスク、worklog、ADR	業務として受け入れるか
Demo4 候補			
Demo5 介入圧縮	現場CI	同じセンサーの自動実行	マージ前に止める

デモ：リリース候補として揃ったもの、現場で足すもの



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

docs/specs/order-estimate.md

last-updated: 2026-06-26

送料無料閾値: 7,000円

6,999円 => 送料500円

7,000円 => 送料0円

docs/decisions/ADR-0002

Context: CR-2026-06-26

Decision:

- thresholdはpricing.js

- 判定は

freeShippingPolicy

- response形式は維持

docs/ai/worklog.md

依頼: CR-2026-06-26

やったこと:

threshold/test/spec/ADR

DR

発見: 旧specは3,000円

残リスク: UI/改定日/過去注文

ここまで揃ったもの

受け入れ条件、テスト、
policy-boundary、spec、ADR、worklog

現場でまだ足すもの

CI、事業側確認、本番相当データ、非機能、監視、ロールバック

デモ：人間が自律ループの入口を設計する



Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

CR front-matter

status: in_progress

acceptance: [test, lint, typecheck,
policy-boundary, api-response,
change-package]

expected_behaviors:

6999 => 送料500 / 7000 => 送料0

required_artifacts:

spec / ADR / worklog / test

max_iterations: 5

AGENTS.md ループプロトコル

1. status: in_progress のCRを読む

2. loop-stateに計画を書く

3. failing test -> 最小実装

4. 毎回 npm run sensors

5. 赤なら原因と次方針を記録

6. completion-reportを書いて停止

停止条件: max_iterations / 同一赤 /
scope外

✓ ポイント

新しく追加されたのは、作業を始める条件(入口CR)、繰り返し作業のルール(ループ規約)、強制的に止める条件(停止条件)、終わったときの報告書(出口report)の4つ。

デモ：完了条件と停止条件で途中介入を減らす



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

そのまま貼るプロンプト

AGENTS.mdに従い、CR进行处理してください。

- expected_behaviorsとrequired_artifactsを残作業リストにする
- 赤sensor、原因、次方針をloop-stateへ更新する
- policy-boundaryを壊さない
- completion-report完成時だけ人間へ返す

AIが更新する進行記録
docs/ai/loop-state.md

- iteration
 - red sensor
 - cause summary
 - next plan
- docs/ai/completion-report.md
- sensor結果
 - 残リスク
 - 人間が見る点

AIが更新するもの

spec/ADR/worklog/test/CR status

AIが生成するもの

loop-state/completion-report/escalation-report

出来たこと

途中介入を減らせる

不足してること

センサーが薄い領域は守れない

デモ：入口と出口に、人間の責任は残る



Demo0

開始前

Demo1

変更

Demo2

挙動

Demo3

構造

Demo4

候補

Demo5

介入圧縮

completion-report

- CR-ID:
CR-2026-06-26-free-shipping-threshold
- 反復回数: 3
- 全sensor結果: green
- 変更ファイルと一行理由
- 残リスク
- 人間が確認すべき点: 3項目以内

位置

見るもの

入口

CRの意図、対象外、受け入れ条件

途中

原則見ない。停止条件ならescalation-report

出口

diff、sensor結果、残リスク、記録

デモ：テスト自動化の考え方



✓ コードを書くAIに対して、人間は仕様と検証の設計を担う。

Demo0
開始前

Demo1
変更

Demo2
挙動

Demo3
構造

Demo4
候補

Demo5
介入圧縮

狭義のテスト自動化

テストコードを書く作業を自動化する。

今日のテスト自動化

AIがコードを変更したあとも、「壊れていないか」を確認する材料(テスト結果やチェック結果)を、自動で集めてくれる。

自動化の振り返り

Demo3

Demo4

Demo5

責務境界をセンサー化する

作ったハーネスを使って実際の仕様変更をリリース候補にする。

センサー実行、修正、記録、完了報告までAIが反復できるようにする

デモ：冒頭の足りないものはどう埋まったか



	内容	開始時点	デモ後
Demo0 開始前	既存挙動の固定	ない	characterization testで固定
Demo1 変更	新仕様の受け入れ条件	ない	CRで受け入れ条件化
Demo2 挙動	構造チェック	ない	policy-boundary sensorで検出
Demo3 構造	AIへの作業規約	ない	AGENTS.mdで継続
Demo4 候補	判断記録	ない	spec / ADR / worklogへ還流
Demo5 介入圧縮	人間が確認・判断する出口	未定	completion-reportで絞る

01 はじめに

02 AI開発の全体像

03 テスト自動化

04 デモ

05 まとめ

06 最後に

最後に：明日から出来る事



最初から自律ループを目指さず、まず判断できる入口と出口を作ってみる。

- ✓ 変更依頼を「CR」として、短く書いてみる。
- ✓ AIに影響範囲とテスト観点表を先に出させる。
- ✓ 人間が固定すべき既存挙動の範囲を決める。
- ✓ 採用した観点だけをテスト化して実行する。
- ✓ 判断理由と残リスクをworklogに1行残す。

| 最後に：今日の結論



- ✓ AIに任せるのではなく、AIに安全に任せられる状態を作る。
- ✓ AIが直せるかどうかだけを見ても、現場の不安は消えない。
- ✓ 必要なのは、Guides / Sensors / Steeringでリリース判断の根拠を作ること。
- ✓ テスト自動化は、その最初の一步になる。

| 最後に：質疑応答とアンケート



- ✓ 何でもお気軽にご質問ください！
- ✓ 迅速に次回の勉強会の企画に役立てたいので、本日中にアンケートのご記入をお願いします！

